

Foundations of 4D/6D Object Capture for Virtual Environments

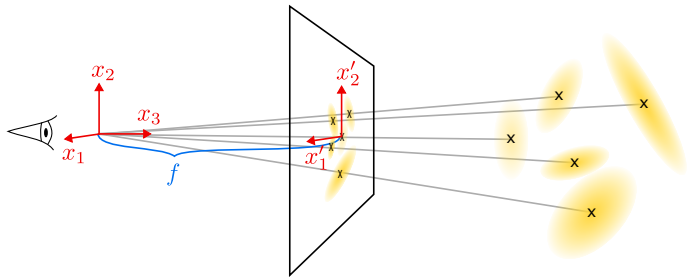
Inverse Kinematics



March 11, 2026



→ **Gaussian Splatting**



Today: Inverse Kinematics

Thanks to

Prof. Dr. Björn Krüger (bkrueger@uni-bonn.de)

for providing information from his lecture
MA-INF 2202 Computer Animation

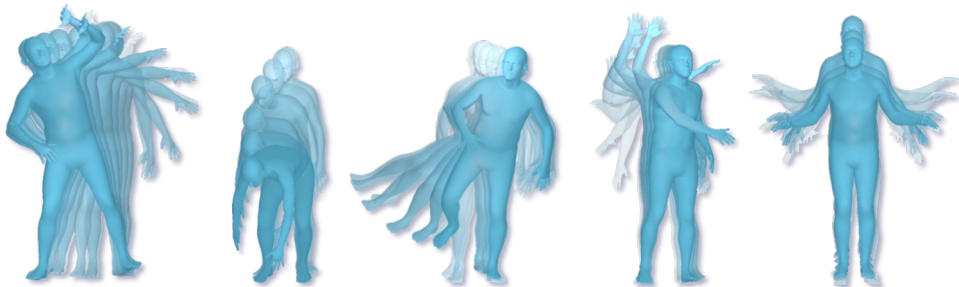
<https://digital-health-bonn.de>



We want to represent motion as changing positions over time t

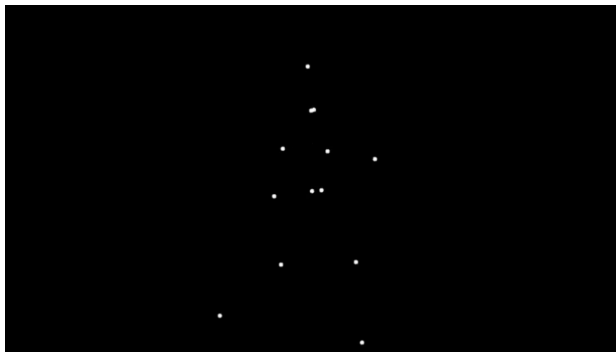
- ▶ Either discrete or continuous time desirable depending on the task
- ▶ Lots of vertices for high-quality meshes

→ Not efficient for e.g. $\sim 10k$ vertices at 30 fps, some separation like in SMPL is needed



Question: How much information do we really need?

Idea: Conduct psychological experiment → Johansson Experiment



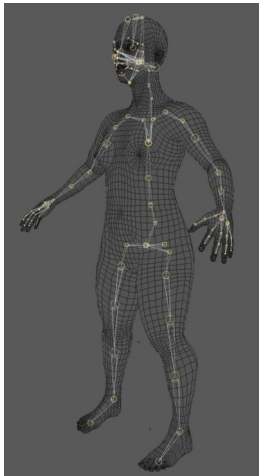
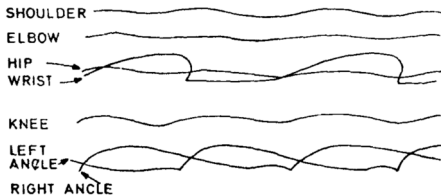
<https://www.youtube.com/watch?v=rEVB6kW9p6k>

Question: How much information do we really need?

→ Usually very little!

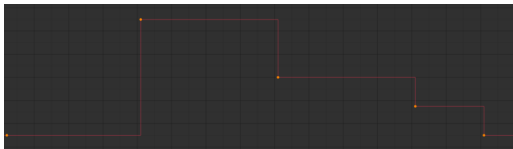
- ▶ Separate soft body parts (shape) from rigid ones (skeleton + pose)
- ▶ Not restricted to human models
- ▶ Easy individual manipulation possible

→ Very efficient to “just” capture ~ 20 joints at 30 fps

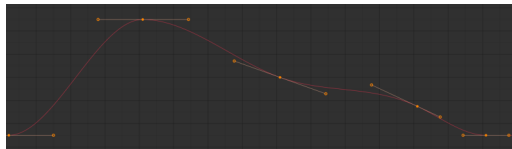


Efficiency can be further increased by only capturing keyframes:

- ▶ Fit interpolation function to captured motion
- ▶ Choose function depending on needs



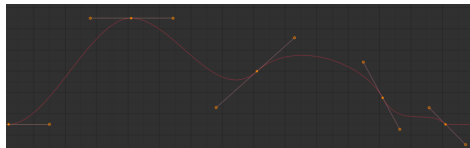
Constant values



Bézier curves 1



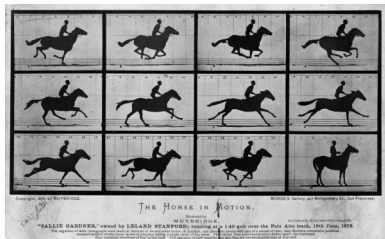
Linear interpolation



Bézier curves 2

There are different types of systems to capture motions:

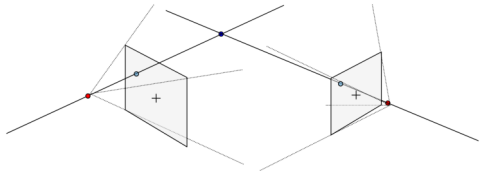
- ▶ Photography
- ▶ Mechanical systems
- ▶ Acceleration sensors
- ▶ Acoustic systems
- ▶ Magnetic systems
- ▶ Optical systems



Optical systems are quite popular

Idea:

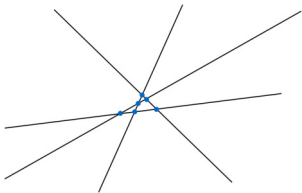
- ▶ Use markers attached to subject → easy to detect
- ▶ Observe markers from multiple view points
- ▶ Triangulate using known camera parameters
→ Same measurement principle as for Stereo Cameras



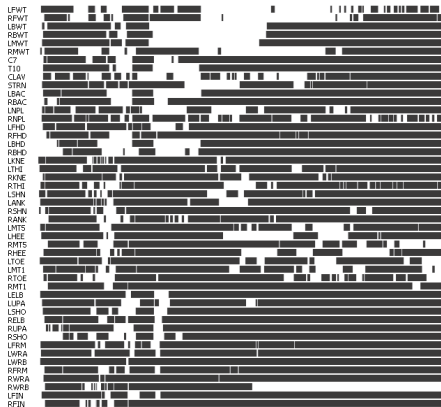
Optical systems are quite popular

Problems:

- ▶ Limited visibility
- ▶ Misalignment



- ▶ Misidentification of markers
- ▶ General detection errors/uncertainties



Continuity diagram of marker visibility

Nowadays, *markerless tracking* becomes increasingly popular

→ Less precise but also less preparation

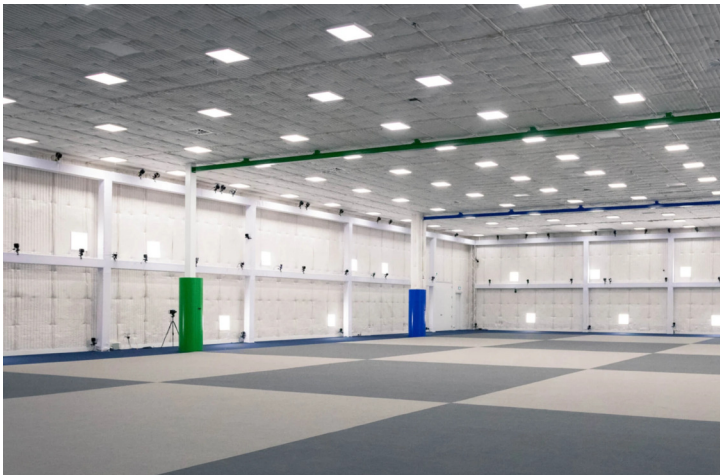
Industry: Captury, Microsoft Kinect



Academia: OpenPose

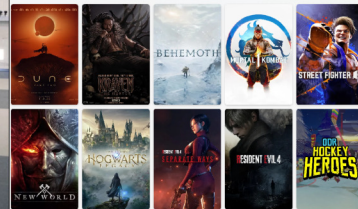


In industry, capturing halls for large scale productions are often used (e.g. 20m × 50m × 10m)



Recent Projects

Incredible partnerships make for incredible work.

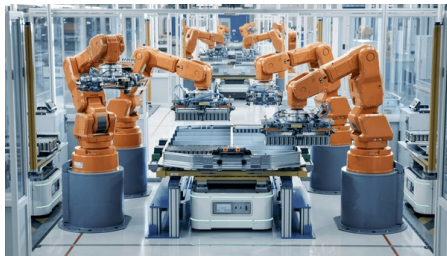


Before working with motions, we first need to learn about Kinematics → originates from **Robotics**

Exemplary Tasks:

- ▶ Grab, move and release objects
- ▶ Assemble parts during construction
- ▶ Write text or draw images

Forward Kinematics $\longleftrightarrow$ Inverse Kinematics



Task: Compute key point positions and orientations s based on controllable parameters θ

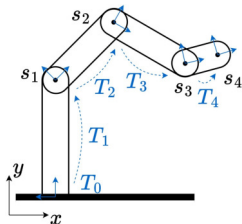
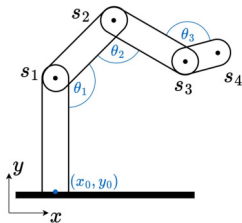
$$s = f(\theta)$$

→ Key points treated as coordinate systems that are translated and rotated compared to the last key point

Thus, location of key points are determined by using transformation matrices T :

$$s_i = T_i(\theta_i) \cdot T_{i-1}(\theta_{i-1}) \cdot \dots \cdot T_0(\theta_0) \cdot s_0$$

- ▶ Joints are connected via rigid parts
- ▶ Lengths do not change, only angles θ do



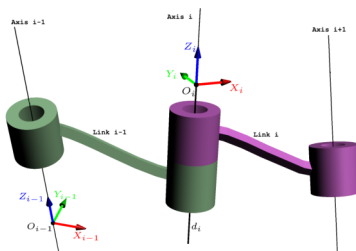
Idea: Restrict to hinges and sliding joints

- Define z -axis as the joint's rotation axis
- Define x -axis as pointing away from direction to the previous joint

→ Transformations are combinations of **rotations** and **translations** in x and z :

$$R_z(\theta) = \begin{pmatrix} \cos(\theta) & -\sin(\theta) & 0 & 0 \\ \sin(\theta) & \cos(\theta) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad T_z(d) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & d \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$R_x(\alpha) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos(\alpha) & -\sin(\alpha) & 0 \\ 0 & \sin(\alpha) & \cos(\alpha) & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad T_x(a) = \begin{pmatrix} 1 & 0 & 0 & a \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$



3 of 4 parameters (d, α, a) are *fixed*
1 of 4 parameters (θ) is DoF

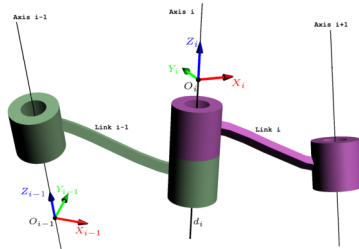
Observation: In contrast to the general matrices, R and T applied for the *same axis* commute:

$$\begin{aligned}R_z(\theta) \cdot T_z(d) &= T_z(d) \cdot R_z(\theta) \\ R_x(\alpha) \cdot T_x(a) &= T_x(a) \cdot R_x(\alpha)\end{aligned}$$

Caution: Due to

- ▶ the above commutation property ...
- ▶ ... and different conventions for defining the x -axis

the final transformation $T_{i \rightarrow i+1}$ between joints can look differently in books and papers!



Full transformation to switch from coordinates system of i to $i + 1$:

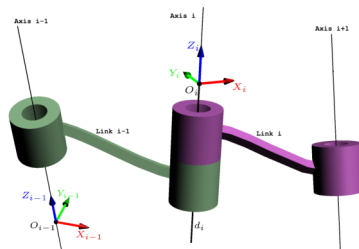
$$\begin{aligned} T_{i \rightarrow i+1} &= \left(R_x(\alpha_i) \cdot T_x(a_i) \cdot R_z(\theta_i) \cdot T_z(d_i) \right)^{-1} \\ &= R_z(-\theta_i) \cdot T_z(-d_i) \cdot R_x(-\alpha_i) \cdot T_x(-a_i) \end{aligned}$$

Order of steps (for moving the coordinate system):

1. $T_z(d_i)$: Move origin of $i - 1$ "up" to common link between axes
2. $R_z(\theta_i)$: Align x -axis with link direction
3. $T_x(a_i)$: Move origin "sideways" to new location in i
4. $R_x(\alpha_i)$: Align z -axis to new desired joint orientation

Reminder: Coordinate transformation is inverse to movement of coordinate system

With $d_i = a_i = 0$ we can overlay joints at the same location
→ Build other types of joints, e.g. *spherical joints*



<https://youtu.be/rA9tm0gTln8>

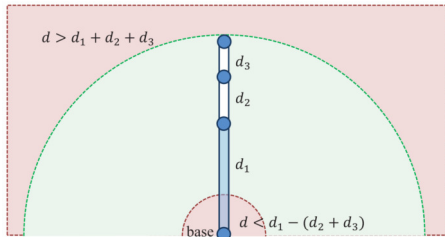
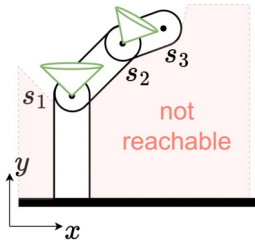
Observation: Joints often have some constraints

- ▶ Bend only in one direction (like an elbow or a knee)
- ▶ Slide along a rail with given end points
- ▶ Limited range

$$\theta_{\min} \leq \theta \leq \theta_{\max}$$

- ▶ Dependencies on other joints

→ Some areas may not be reachable



Task: Which parameters θ^* do we need to reach a target position (and orientation) t ?

$$\theta^* = f^{-1}(t)$$

→ Inverse Problem to Forward Kinematics

Idea: Linearize equation around current state $s = f(\theta)$:

$$t = f(\theta^*)$$

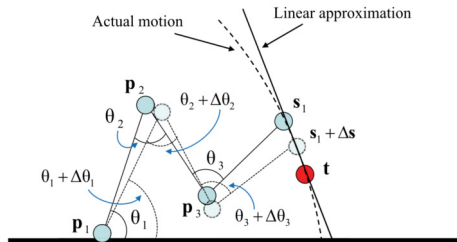
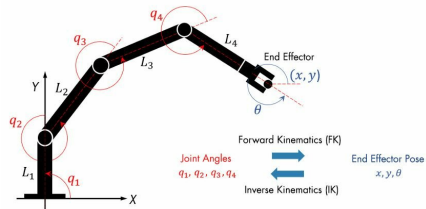
$$\approx s + J(\theta) \cdot \Delta\theta \quad \text{with } J_{ij}(\theta) = \frac{\partial s_i}{\partial \theta_j}$$

We can now rearrange this approximation to get

$$\Delta\theta = J^{-1}(\theta) \cdot (t - s)$$

So, we can iteratively update our state by

$$s_{\text{new}} = s + \Delta s = f(\theta + \Delta\theta)$$



Problem: We cannot directly compute $\Delta\theta$ this way!

1. Jacobian is usually not invertable (not even square)

$$\Delta\theta^* = \min_{\Delta\theta} \|J(\theta) \cdot \Delta\theta - (t - s)\|^2$$

→ Solve **Least Squares** problem instead

2. Solving for $\Delta\theta^*$ still unstable near impossible configurations → Jittering or complete failure

$$\Delta\theta^* = \min_{\Delta\theta} \|J(\theta) \cdot \Delta\theta - (t - s)\|^2 + \lambda \|\Delta\theta\|^2$$

→ Solve **Damped Least Squares (Levenberg–Marquardt algorithm)** problem for regularization

→ Damping controllable via $\lambda \in \mathbb{R}_{\geq 0}$: $\lambda \rightarrow \infty$ leads to $\Delta\theta \rightarrow 0$

Simple Example: Here, we use world space angles for easier formulas:

$$\mathbf{s}_0 = (x_0, y_0)^T$$

$$\mathbf{s}_1 = \mathbf{s}_0 + l_1 \cdot (\cos(\theta_1), \sin(\theta_1))^T$$

$$\mathbf{s}_2 = \mathbf{s}_1 + l_2 \cdot (\cos(\theta_2), \sin(\theta_2))^T$$

$$\mathbf{s}_3 = \mathbf{s}_1 + l_3 \cdot (\cos(\theta_3), \sin(\theta_3))^T$$

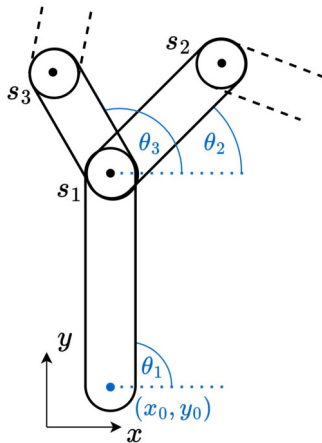
For a freely moving chain, we get (considering the first 4 vertices)

$$\boldsymbol{\theta} = (x_0, y_0, \theta_1, \theta_2, \theta_3)^T \rightarrow \mathbf{J}(\boldsymbol{\theta}) \in \mathbb{R}^{8 \times 5}$$

The derivatives are

$$\frac{\partial \mathbf{s}_i}{\partial x_0} = (1, 0)^T, \quad \frac{\partial \mathbf{s}_i}{\partial y_0} = (0, 1)^T$$

$$\frac{\partial \mathbf{s}_i}{\partial \theta_j} = \begin{cases} l_j \cdot (-\sin(\theta_j), \cos(\theta_j))^T & \text{if } j = i \text{ or } j \text{ is parent of } i \\ (0, 0)^T & \text{otherwise} \end{cases}$$



Problem: Incorporating constraints to the Jacobian method is not easy

→ Use other types of solvers

Linear Programming

Define weights $c \in \mathbb{R}^n$ and solve

$$\min_{x \in \mathbb{R}^n} \langle c | x \rangle$$

subject to

$$A_{\text{eq}} x = b_{\text{eq}}$$

$$A_{\text{ub}} x \leq b_{\text{ub}}$$

$$l \leq x \leq u$$

Quadratic Programming

Define weights $c \in \mathbb{R}^n, Q \in \mathbb{R}^{n \times n}$ and solve

$$\min_{x \in \mathbb{R}^n} \frac{1}{2} x^T Q x + \langle c | x \rangle$$

subject to

$$A_{\text{eq}} x = b_{\text{eq}}$$

$$A_{\text{ub}} x \leq b_{\text{ub}}$$

$$l \leq x \leq u$$

→ Every Least Squares problem can be reformulated in this form

Idea: Compute angles of kinematic chain in a heuristic manner

→ **F**orward **A**nd **B**ackward **R**eaching **I**nverse **K**inematics (FABRIK)¹

Iterate between the following steps:

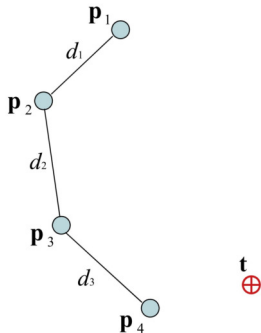
1. Backward Step

- ▶ Place last segment at the **target t** and point it back to where it came from
- ▶ Repeat for second last segment until reaching the root

2. Forward Step

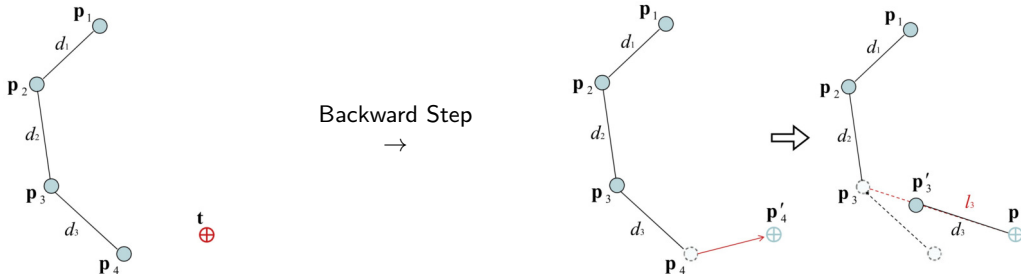
- ▶ Place first segment at its **origin p_1** and point it back to where it came from *after the backward step*
- ▶ Repeat for second segment until reaching the end effector

Stop algorithm if target is reached or no change happened



¹A. Aristidou and J. Lasenby. "FABRIK: A fast, iterative solver for the Inverse Kinematics problem". In: *Graphical Models* (2011).

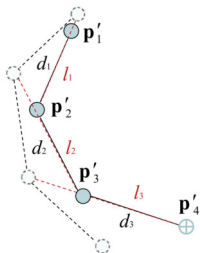
Example: Iteration 1



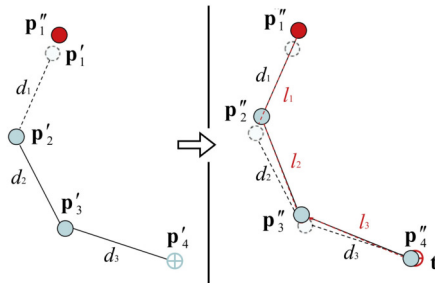
1. Backward Step

- ▶ Place last segment at the **target t** and point it back to where it came from
- ▶ Repeat for second last segment until reaching the root

Example: Iteration 1



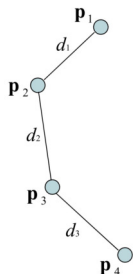
Forward Step



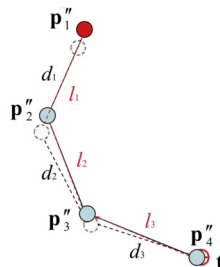
2. Forward Step

- ▶ Place first segment at its **origin** p_1 and point it back to where it came from *after the backward step*
- ▶ Repeat for second segment until reaching the end effector

Example: Iteration 1



Backward Step + Forward Step
→



Observations:

- Position of end effector p_4 is closer to desired target t
- Both backward and forward step are needed → origin p_1 should stay the same

<https://youtu.be/Ihp6t0CYHug?t=43>

<https://editor.p5js.org/rjgilmour/sketches/2sbLGqpuZ>

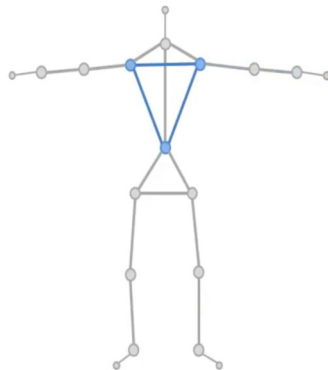
Advantages:

- ▶ Fast and easy to implement
→ No transformations or angles, just positions and vectors
- ▶ Stable even with impossible targets
- ▶ Easy to incorporate constraints

Disadvantages:

- ▶ Not optimal in any sense
→ e.g. least change in angles
- ▶ Depends on ordering, so not trivial for
 - ▶ Branches
 - ▶ Closed loops
 - ▶ Combinations of these

→ Detect and treat these in special cases²



²A. Aristidou, Y. Chrysanthou, et al. "Extending FABRIK with model constraints". In: *Computer Animation and Virtual Worlds* (2016).

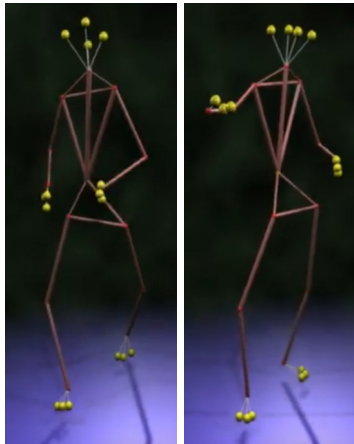
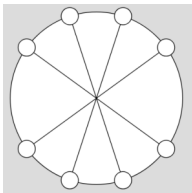
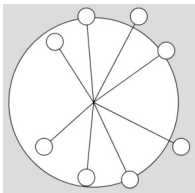
Idea: Define target locations/orientations for some joints

- ▶ ... Partial motion capturing
- ▶ ... By defining an objective
- ▶ ... By hand

→ Let **Inverse Kinematics** adjust the remaining joints

This can even be fully automated: *Procedural Motion Generation*

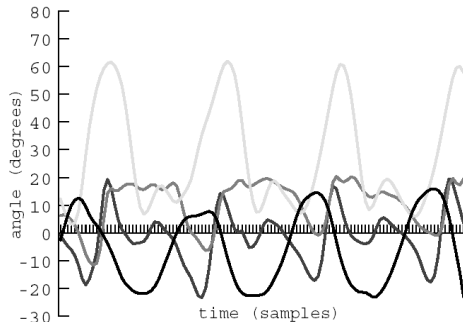
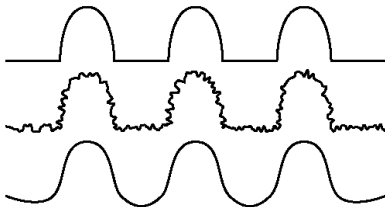
- ▶ Shift target continuously or depending on some condition



Various goals can be defined:

- ▶ Remove noise from the captured data
- ▶ Make motion periodic
- ▶ Change the animation speed (irregularly)
- ▶ Align or blend two motion sequences in time
- ▶ Change the motion of a single body part
- ▶ ...

Preserve fine (high-frequency) details. Not like this:

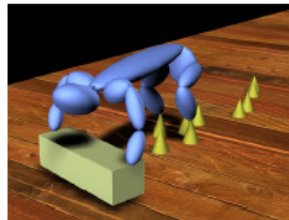
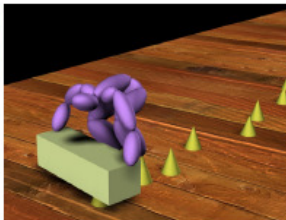
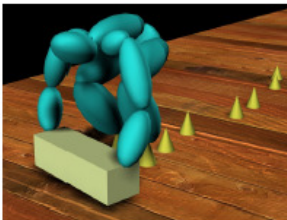
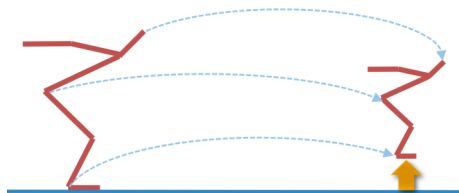


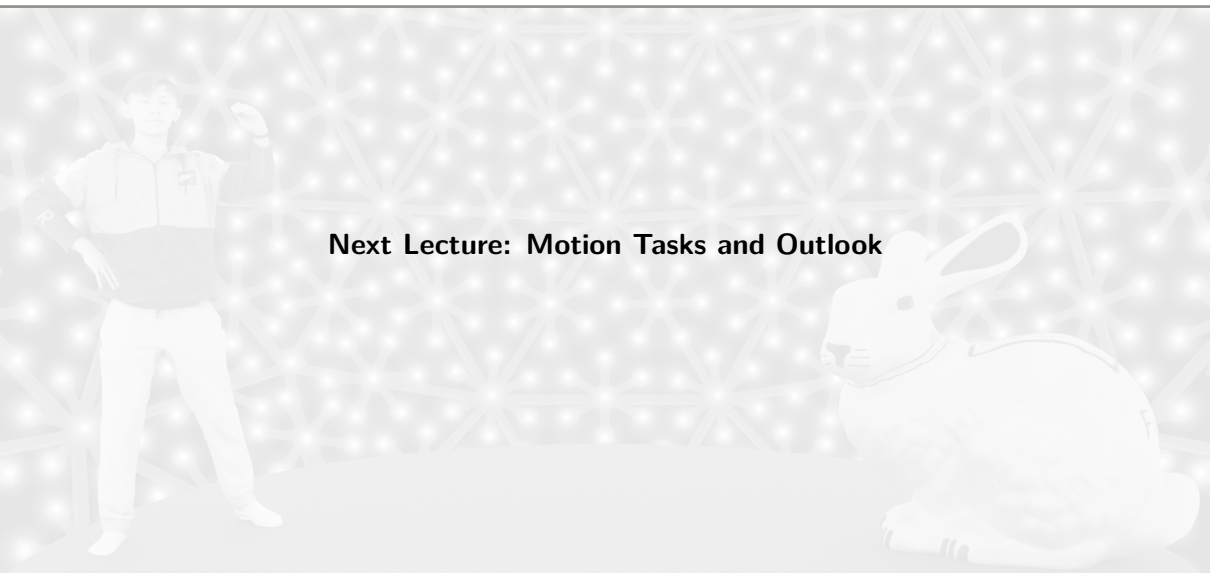
Captured walking animation

Edit the motion while preserving as many constraints as possible:

- ▶ All joint constraints
- ▶ Geometry constraints (e.g. feet on the ground)
- ▶ Physical constraints (e.g. balance and momentum)
- ▶ Overall realistic looking motion

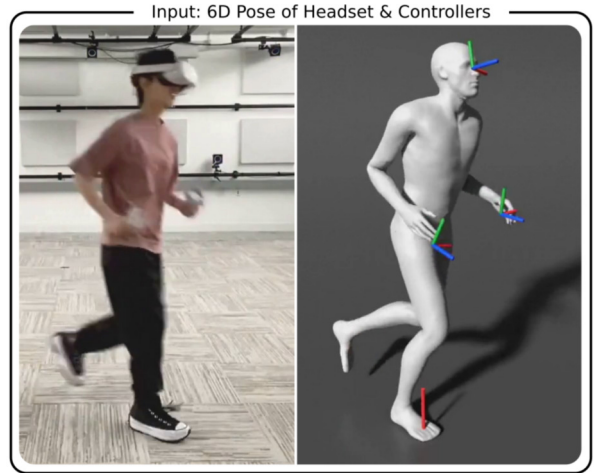
→ Discard less important constraints





Next Lecture: Motion Tasks and Outlook

→ Motion Tasks and Outlook



- [1] A. Aristidou, Y. Chrysanthou, and J. Lasenby. “Extending FABRIK with model constraints”. In: *Computer Animation and Virtual Worlds* (2016).
- [2] A. Aristidou and J. Lasenby. “FABRIK: A fast, iterative solver for the Inverse Kinematics problem”. In: *Graphical Models* (2011).